Policy of Truth Implementation and Governance Architecture Report: A Forensic Audit and Strategic Roadmap for Final Phase Consolidation

1. Executive Summary: The Inflection Point of Governance as Code

The Stela repository stands at a critical architectural juncture. Having transitioned from a loose collection of operational scripts to a constitutionally governed environment under the **Policy of Truth (PoT)**, the system is now grappling with the friction inherent in moving from "Post-Hoc Validation" to "Correct-by-Construction" (CbC) methodologies. This report serves as a comprehensive forensic audit of the implementation status, evaluating the efficacy of the recent "Phase 2" and "Phase 3" rollouts and identifying the structural legacy debt that threatens the integrity of the "Final Phase."

The central thesis of this analysis is that while the **Physical Laws** and **Axioms** defined in PoT.md have been successfully codified into the active context and largely enforced by the new immutable workflows, the execution layer—specifically the ops/bin/ binaries and ops/lib/scripts/ automation—retains brittle, legacy logic patterns. These patterns, characterized by text introspection and dual-use artifacts, fundamentally violate the **Filing Doctrine** (Section 1.1 of PoT), which mandates a strict segregation between ops/ (Run) and docs/ (Explain).[1]

The successful deployment of **DP-OPS-0065**, which introduced the ops/bin/draft binary and the canonical ops/src/surfaces/DP.md.tpl template, marks a definitive victory for the CbC doctrine.[1] It has proven that operational security is maximized when the "work surface" (TASK.md) is generated from immutable definitions rather than hand-authored prose. However, the persistence of "Factory Ledgers" in opt/_factory/ and the "Mining" logic in ops/bin/prune represents a critical vulnerability. The current state is a hybrid architecture: the governance is absolute, but the enforcement machinery is still shedding the weight of heuristic parsing.

This report proposes a rigorous **Definition Layer Migration**, moving all generative templates to a dedicated ops/src/definitions/ namespace to decouple structure from content. Furthermore, it outlines a **Code Hygiene Sweep** to refactor ops/bin/prune and library scripts into deterministic, low-complexity utilities that read explicit inputs rather than parsing documentation. This strategy is not merely a cleanup; it is the necessary architectural

conclusion to the PoT implementation, ensuring that the repository's "Physical Laws" are enforced by physics, not just policy.

---

2. Theoretical Framework: The Policy of Truth as System Physics

To evaluate the rollout's success, we must first dissect the theoretical underpinnings of the Policy of Truth. The PoT is not a passive "style guide"; it is defined as the "single constitutional document for Stela governance".[1] Its preamble asserts a doctrine of active prevention: "The system prevents action that violates policy. It does not request compliance".[1] This distinction is vital for analyzing the recent Dispatch Packets.

2.1 The Filing Doctrine and Axiomatic Constraints

The **Filing Doctrine** (PoT Section 1.1) is the primary spatial law of the repository. It dictates the functional role of every byte based on its location:

- **ops/ (Run):** This is the domain of executable logic. It includes binaries, manifests, and automation. Crucially, artifacts here must be machine-readable and deterministic.
- **docs/ (Explain):** This is the domain of human understanding. It includes manuals (MANUAL.md) and rationale (PoT.md). Artifacts here are prose-heavy and should never be critical path dependencies for execution logic.
- **projects/ (Work):** The payload layer where value is created.
- **storage/ (Trash):** Ephemeral, local-only artifacts that are never canon.

The forensic analysis in Section 3 will reveal that the current implementation of ops/bin/prune violates this doctrine by treating docs/ as a runtime dependency.[1]

2.2 The Axiom of "System" and "Truth"

Two axioms from PoT Section 1.2 are particularly relevant to the current state:

1. **System:** "Everything the system proves is actually true." This axiom demands that the system's state must be verifiable via executable proofs (receipts), not just claims. The **Proof of Work (PoW)** ledger is the physical manifestation of this axiom.
2. **Truth:** "Primitive predicate governed by specific rules rather than a mere definition." Truth in Stela is not abstract; it is file compliance. A file is "true" if it matches its definition in the registry and passes all lints.

The transition to **Correct-by-Construction (CbC)** is the operationalization of these axioms. In a CbC system, invalid states are unrepresentable. A user cannot create a malformed Dispatch Packet because the draft tool only produces valid ones. A user cannot ship code

without a PoW entry because the prune tool blocks the cleanup of evidence.[1]

---

3. Forensic Analysis of the Implementation Rollout (Phase 2 & 3)

The rollout of the Policy of Truth has been executed through a series of "Dispatch Packets" (DPs), each addressing specific failure patterns identified in the "Policy of Truth Implementation Plan".[1] By analyzing the **Proof of Work (PoW)** and **State of Play (SoP)** ledgers, we can reconstruct the trajectory of this rollout and identify the specific failure modes that persist.

3.1 DP-OPS-0063: The Closeout Ledger Compliance (Phase 1 Stabilization)

Objective: The primary goal of DP-OPS-0063 was to enforce the axiom that "Routing Closeout failure is a system failure state".[1] Historically, the "Closeout" phase (Section 4 of the old TASK schema) was perceived by agents as a "post-work" activity, leading to frequent omissions where code shipped without a corresponding update to the State of Play (SoP.md) or Proof of Work (PoW.md) ledgers.[1]

Execution Analysis: To remedy this, the "Closeout" phase was renumbered from Section 4 to **Section 3.5** in TASK.md, structurally integrating it into the active Dispatch Packet scope.[1] This seemingly minor semantic shift had profound implications for the Large Language Model (LLM) completion bias; by making the closeout a subsection of the active task (3.x), the model views it as a mandatory dependency for atomic completion rather than an optional aftermath.

The "Failure" that Proved the System: The most critical insight from DP-OPS-0063 comes from its "failure." The RESULTS artifact for this DP documents that the final hygiene command, ./ops/bin/prune --dp=DP-OPS-0063 --scrub, failed with **Exit Code 1**.[1] The error message was explicit: **"SAFETY VIOLATION: Uncommitted Results artifact detected."**

Under a traditional "Post-Hoc Validation" model, this would be a bug—the tool failed to clean up. But under the **Policy of Truth**, this was a **Positive Proof**. The system's safety logic correctly identified that the operator had not yet committed the evidence (RESULTS.md) to the git history. To prevent the destruction of evidence (the local handoff files) before it was secured in the immutable ledger (git), the tool refused to run. This incident confirms that the **Safety Logic** embedded in ops/bin/prune is functioning as a constitutional guardrail, enforcing the "Action" axiom that "deception or incoherence are inherently destabilizing".[1]

3.2 DP-OPS-0064: Structural Restructuring (Phase 2)

Objective: This packet executed the physical migration of library content from docs/library/

to opt/_factory/ to implement **Pattern 2** (Context Hazard Enforcement) of the Implementation Plan.[1]

Context Hazard Mitigation: The **Context Hazard** axiom [1] states: "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure." In the previous architecture, these libraries resided in docs/. Because docs/ is the "Explain" layer, agents frequently globbed docs/**/*.md into the context window, accidentally ingesting hundreds of pages of instruction manuals that confused their operational directives.

By moving these libraries to opt/_factory/, the system utilized the **Filing Doctrine** as a firewall. opt/ is explicitly defined as "Isolate (JIT-only library content excluded from global context)".[1] This physical isolation means that standard documentation globs (docs/*.md) strictly exclude the agent instructions. The implementation of ops/bin/compile further hardened this by generating explicit manifests (ops/lib/manifests/OPS.md) from templates (ops/src/manifests/*.md.tpl), ensuring that the repository's "memory" is a compiled artifact rather than a fluid runtime calculation.[1] This successfully resolved **Pattern 5** (Manifest Synthesis Complexity).[1]

3.3 DP-OPS-0065: Immutable Workflow Adoption (Phase 3)

Objective: The deployment of ops/bin/draft and the canonical ops/src/surfaces/DP.md.tpl template.[1]

The Shift to Correct-by-Construction: Prior to DP-OPS-0065, TASK.md was a mutable surface. Agents were expected to "copy-paste" the correct headers and sections. This led to **Pattern 4** (DP Lint Structural Enforcement) failures, where valid work was rejected because an agent missed a newline or capitalized a header incorrectly.[1] tools/lint/dp.sh relied on brittle regular expressions (regex) to "spell-check" the structure.

With DP-OPS-0065, the workflow became immutable. The ops/bin/draft binary acts as a factory, instantiating the DP.md.tpl template into the active work surface. The user does not write the structure; the system builds it. Crucially, the tools/lint/dp.sh script was refactored to enforce a specific **SHA-256 hash** of the canonical template.[1] If the template hash in the lint script does not match the file on disk, the system halts. This shifts validation from "Does this text look right?" (Regex) to "Is this the exact authorized structure?" (Hash).

Current Status: The rollout of DP-OPS-0065 was successful. The RESULTS indicate that the ops/bin/draft tool and the new template were staged and verified, and the diff accounting confirms the refactor of tools/lint/dp.sh to enforce structure-hash validation.[1]

3.4 Summary of Rollout Status

The implementation of the Policy of Truth is effectively in its final operational phase. The

"Physical Laws" are established, and the "Immutable Workflow" is live.

- **Pattern 1 (Closeout Schism): Resolved** via DP-OPS-0063 (renumbering).
- **Pattern 2 (Context Hazards): Resolved** via DP-OPS-0064 (physical isolation to opt/).
- **Pattern 4 (DP Structure): Resolved** via DP-OPS-0065 (immutable templates).
- **Pattern 5 (Manifests): Resolved** via DP-OPS-0064 (compiled manifests).

The remaining friction points are vestiges of the previous era—specifically **Pattern 3** (Reset-Task Sequencing) and **Pattern 6** (Allowlist Validation). These patterns are failing because the tools that enforce them (prune) still rely on parsing text files rather than reading explicit definitions.

---

4. Legacy Artifact Analysis: The Task Surface and Prune Binary

To accurately determine the path forward, we must perform a deep inspection of the specific artifacts flagged in the user query: docs/ops/specs/surfaces/task.md and ops/bin/prune. The user asks: "Is template located in this file docs/ops/specs/surfaces/task.md, now legacy?" and "Need to do a sweep of all binaries... and make less brittle."

4.1 Is docs/ops/specs/surfaces/task.md Legacy?

Verdict: YES, functionally legacy and structurally non-compliant.

Evidence:

1. **Dual Source of Truth:** This file currently acts as both the *Human Specification* for the TASK surface and the *Machine Source* for the ops/bin/prune --reset-task command.[1] This violates the **SSOT** (Single Source of Truth) axiom.[1] A file cannot be both documentation (Explain) and code (Run).
2. **Delimited Extraction:** The file contains markers like and.[1] This pattern—embedding code/templates inside documentation blocks—is the hallmark of the "Post-Hoc Validation" era. It forces the prune binary to act as a text scraper, using awk or sed to mine the template.
3. **Conflict with Draft:** With the arrival of ops/bin/draft and ops/src/surfaces/DP.md.tpl in DP-OPS-0065 [1], the generation of the TASK.md content has moved to a dedicated template pipeline.
    - ops/bin/draft generates the DP section.
    - ops/bin/prune --reset-task generates the *rest* of the TASK.md file (headers, Session State, pointers).
    - This split ownership (part template, part doc-extraction) creates ambiguity. If the documentation is edited for clarity, the reset-task command might break or corrupt

the task dashboard.

Conclusion: The template content embedded within docs/ops/specs/surfaces/task.md must be extracted to a new, explicit template file: ops/src/surfaces/TASK.md.tpl. The documentation file should then reference the template, rather than containing it. This renders the *template portion* of the file legacy, though the *specification text* remains valid documentation.

4.2 Forensic Analysis of ops/bin/prune

Current State: The prune binary is described as having "over 600 lines of defensive coding".[1] It performs three distinct roles:

1. **Garbage Collection:** Removing old artifacts based on age or DP ID.
2. **Ledger Archival:** Truncating SoP.md and PoW.md and moving entries to archive files.
3. **State Reset:** Rewriting TASK.md from the embedded template in the docs.

Legacy Patterns & Brittleness:

- **Fragile Introspection:** The reset-task function uses regex/awk delimiters to parse the doc file. This logic is inherently brittle. A user correcting a typo in the documentation comments could break the system's ability to reset its state. This is a violation of the **Axiom of System Stability**.
- **Safety Logic Coupling:** The "Results Guard" logic [1] is embedded directly in the prune script. While the *intent* is correct (preventing data loss), the *implementation* couples the action of pruning with the policy of verification. In a CbC architecture, safety checks should be shared library functions invoked by the tool, not inline logic that bloats the script.
- **Ledger Parsing:** The logic to count entries (e.g., "keep most recent 30 entries") likely involves regex matching on header lines (## YYYY-MM-DD...). This is susceptible to breakage if the header format changes even slightly (e.g., changing the date separator or spacing).

Refactor Opportunity:

ops/bin/prune represents the largest accumulation of technical debt in the ops/ namespace. It should be reduced to a "dumb" executor.

- **Input:** It should accept a list of files to delete (generated by a separate heuristic or explicitly passed) and a template path to restore.
- **Logic:** It should simply read ops/src/surfaces/TASK.md.tpl and write TASK.md. Zero parsing of documentation required.
- **Reduction:** Moving the logic from "extract from docs" to "read from file" will likely remove 50-100 lines of fragile parsing code.

---

5. Strategic Proposal: The Definition Layer Migration

To fully realize the **Correct-by-Construction** architecture and address the user's request to "move templates... to here ops/src/definitions/*.md.tpl," we must eliminate the "Factory Ledger" pattern currently used in opt/_factory/.

Currently, files like opt/_factory/AGENTS.md act as both a **Registry** (list of agents) and a **Template Source** (containing the blank agent template for new creations).[1] This violates the **Single Source of Truth (SSOT)** axiom. A registry should be a list. A template should be a file. Mixing them invites drift.

5.1 The New Taxonomy: ops/src/definitions/

I propose the creation of a new directory structure: ops/src/definitions/. This directory will house the immutable "Genetic Code" for all system entities (Agents, Tasks, Skills). This completes the Filing Doctrine for the source (src) namespace.

Filing Doctrine Update:

- ops/src/manifests/ -> Compilation sources for Context (Existing).
- ops/src/surfaces/ -> Templates for Interfaces (DP, TASK, PoW) (Existing/Expanded).
- ops/src/definitions/ -> Templates for Entities (Agents, Tasks, Skills) (New).

5.2 Migration Plan: From Factory to Definition

We will migrate the embedded templates out of the markdown ledgers and into explicit .tpl files.

5.2.1 Agent Definition

- **Source:** Embedded block in opt/_factory/AGENTS.md (or hardcoded heredocs in ops/lib/scripts/agent.sh).
- **Destination:** ops/src/definitions/agent.md.tpl
- **Content:** The skeleton structure of an agent, including sections for Provenance, Specialization, Pointers, Authorized Toolchain, and Scope Boundary.
- **Variables:** The template will use standardized tokens (e.g., {{AGENT_ID}}, {{NAME}}, {{SPECIALIZATION}}) rather than hardcoded strings or sed placeholders.

5.2.2 Task Definition

- **Source:** Embedded block in opt/_factory/TASKS.md.[1]
- **Destination:** ops/src/definitions/task.md.tpl
- **Content:** The B-TASK schema, enforcing the presence of Provenance, Orchestration, Execution Logic, and the mandatory Closeout pointer.
- **Variables:** {{TASK_ID}}, {{OBJECTIVE}}, {{DP_POINTER}}.

5.2.3 Skill Definition

- **Source:** Embedded block in opt/_factory/SKILLS.md.[2]
- **Destination:** ops/src/definitions/skill.md.tpl
- **Content:** The S-LEARN schema, including Provenance, Invocation Guidance, and Guardrails.
- **Variables:** {{SKILL_ID}}, {{FRICTION_CONTEXT}}.

5.3 Impact on Tooling (ops/lib/scripts/)

The harvest scripts (agent.sh, task.sh, skill.sh) must be updated to read these new templates. Currently, these scripts likely use "heredocs" (inline text blocks) to generate new files. This is "brittle" because a change to the governance of an Agent requires editing the shell script.

- **Current Logic (Legacy Pattern):**
  Bash
  cat <<EOF > "storage/archives/agents/$name.md"
  # Agent Draft

... (hardcoded strings)...

EOF
```

- **New Logic (CbC Pattern):**
  Bash
  template="ops/src/definitions/agent.md.tpl"
  # Use envsubst or sed to fill variables
  sed -e "s/{{NAME}}/$name/g" "$template" > "storage/archives/agents/$name.md"

This change decouples the *logic of creation* from the *structure of the artifact*. If the PoT requires a new section in all Agents (e.g., "Safety Constraints"), we edit the .tpl file, and the script remains untouched. This dramatically reduces the brittleness of the scripts and aligns with the **Reuse-first** axiom.

---

## 6. Code Hygiene and Line Reduction Strategy

The user explicitly requested a "sweep of all binaries/scripts/tools" to begin "serious line reductions" and "make less brittle." The following strategy targets the specific sources of fragility identified in the forensic audit.
```

6.1 The "Fragility" Audit criteria

Fragility in shell scripts usually stems from three sources:

1. **Inline Content Generation:** Using cat <<EOF to generate large blocks of text (violates separation of concerns).
2. **Regex Parsing of Prose:** Using grep/sed/awk to extract data from human-written documents (violates stability).
3. **Implicit State Dependencies:** Assuming a file exists or is in a specific state without verifying (violates determinism).

6.2 Target: ops/bin/prune Refactor

Reduction Strategy:

- **Remove Template Extraction:** Delete the extract_task_template() function and the associated awk logic that mines docs/ops/specs/surfaces/task.md.
- **Direct Read:** Replace the extraction logic with a simple read command: cat ops/src/surfaces/TASK.md.tpl > TASK.md.
- **Simplify Ledger Archival:** The logic to parse dates and move entries in SoP.md is likely complex. Simplify this by standardizing the archive process. If SoP.md exceeds 30 entries, use head and tail to split the file based on line counts or simple grep delimiters, rather than complex date parsing.
- **Decouple Safety:** The "Results Guard" check is vital, but if it is implemented as a verbose inline block, it should be moved to a shared library function (e.g., ops/lib/scripts/governance.sh) or simplified to a single check of the git status of the storage/handoff directory.

Estimated Reduction: Approximately 100-150 lines of complex parsing code can be removed.

6.3 Target: ops/lib/scripts/synthesize.sh Hardening

This script is the "One Truth context engine".[1] **Current Complexity:** It handles recursive includes and globbing to build the context. **Hardening Strategy:**

- Ensure it *only* reads compiled manifests (ops/lib/manifests/*.md).
- Remove any logic that supports runtime globbing (e.g., docs/library/*.md). The compiler (ops/bin/compile) handles globbing. The synthesizer should only handle explicit paths.
- **Impact:** This makes context generation O(1) complexity (read file, print file) rather than O(N) (scan directories, filter, sort). It eliminates the code required to traverse directories and filter files at runtime.

6.4 Target: ops/lib/scripts/{agent,task,skill}.sh Refactor

Refactor Strategy:

- As detailed in Section 5.3, replace all heredocs with template reads.
- **Heuristics:** The heuristics.sh library likely contains git history analysis.[1] Ensure this logic is robust using git log --format to avoid parsing raw output.
- **Registry Interactions:** Ensure that registry updates are atomic. If the scripts currently parse the registry markdown to find the insertion point, verify if this can be simplified by enforcing a sorted append or using a strict table format that is easier to append to.

---

7. Detailed Proposal: Phase 4 Implementation Plan

This proposal outlines the immediate next steps to execute the "sweep" and "migration" requested, completing the transition to the Correct-by-Construction architecture.

7.1 Objective

To complete the Policy of Truth implementation by establishing a formal **Definition Layer**, decoupling artifact structure from generation logic, and reducing the codebase size and complexity of the Ops Kernel.

7.2 Scope

- **New Directory:** ops/src/definitions/
- **New Files (The Definition Layer):**
  - ops/src/definitions/agent.md.tpl
  - ops/src/definitions/task.md.tpl
  - ops/src/definitions/skill.md.tpl
  - ops/src/surfaces/TASK.md.tpl (Extracted from docs)
- **Modified Scripts (The Hygiene Sweep):**
  - ops/bin/prune (Refactor to use templates, remove text mining)
  - ops/lib/scripts/agent.sh (Refactor to use templates)
  - ops/lib/scripts/task.sh (Refactor to use templates)
  - ops/lib/scripts/skill.sh (Refactor to use templates)
- **Modified Docs (The Legacy Cleanup):**
  - docs/ops/specs/surfaces/task.md (Remove embedded template block, replace with pointer)
  - opt/_factory/{AGENTS,TASKS,SKILLS}.md (Remove embedded templates, replace with pointers)

7.3 Implementation Steps

Step 1: Template Extraction (The Lift)

Create the new .tpl files by extracting the content currently embedded in the ledgers and documentation.

- **Action:** Copy the "Embedded Canonical TASK Template" from docs/ops/specs/surfaces/task.md to ops/src/surfaces/TASK.md.tpl. Ensure variables are properly tokenized (e.g., {{DP_ID}}).
- **Action:** Copy the "Promotion Packet Template" or draft skeletons from opt/_factory/*.md to ops/src/definitions/*.md.tpl.

Step 2: The Prune Refactor (The Cut)

Modify ops/bin/prune to abandon its text-mining heritage.

- **Old Logic:** awk '//,//'...
- **New Logic:** cp ops/src/surfaces/TASK.md.tpl TASK.md (plus variable substitution if needed for Session State).
- **Verification:** Run prune --reset-task and verify TASK.md is byte-identical to the desired state.

Step 3: The Harvester Refactor (The Streamline)

Update agent.sh, task.sh, skill.sh.

- Replace internal heredocs with template="$(cat ops/src/definitions/agent.md.tpl)".
- Use simple sed or envsubst for variable replacement.
- This ensures that if we update the definition of an "Agent" in the PoT, we only edit the .tpl file, not the shell script.

Step 4: Documentation Cleanup

- Edit docs/ops/specs/surfaces/task.md. Replace the embedded template block with a pointer: "See ops/src/surfaces/TASK.md.tpl for the canonical template."
- This restores docs/ to its rightful status: **Explain**, not **Run**.

7.4 Success Metrics

- **Line Reduction:** Target >300 lines removed across ops/bin/ and ops/lib/.
- **Fragility Score:** Elimination of grep/awk parsing of markdown files in the critical path of prune and harvest.
- **Compliance:** 100% adherence to Filing Doctrine (Run vs Explain separation).

---

8. Implications and Second-Order Effects

8.1 The "Immutable Agent"

By moving templates to ops/src/definitions, we effectively create a "Type System" for the repository's entities. An agent is no longer "whatever text is in the file"; it is an *instance* of agent.md.tpl. This opens the door for **Schema Validation**: we can write a linter that compares an active agent file against the template to detect drift (e.g., "Agent R-01 is missing the

'Scope Boundary' section defined in the template"). This moves the system closer to a "Typed" governance model.

8.2 Security Hardening

Removing code/templates from docs/ reduces the attack surface. If an attacker (or hallucinating LLM) modifies a documentation file, it no longer breaks the build or the reset tools. The executable logic is strictly contained within ops/, which can be protected by stricter CODEOWNERS policies.

8.3 Governance as Code

This transition completes the shift to **Governance as Code**. The Policy of Truth is not just a text document; it is enforcing itself through the file system structure. The directory layout (ops/src/definitions) physically enforces the distinction between "Abstract Idea" (Template) and "Concrete Instance" (File), mirroring the PoT's distinction between "Truth" (Policy) and "State" (Ledgers).

9. Conclusion

The rollout of the Policy of Truth is effectively in its final operational phase. The foundational laws are sound, and the immutable workflow has secured the active work surface. The remaining friction points are vestiges of the previous era—scripts that act like text parsers rather than system engineers.

The proposal to move templates to ops/src/definitions/ is not just a cleanup; it is the necessary architectural conclusion to the PoT implementation. It resolves the "Dual-Role" artifact violation, eliminates the most brittle code in the repository (prune extraction logic), and solidifies the Correct-by-Construction doctrine.

Recommendation: Approve the **Definition Layer Migration** immediately. This will unblock the final hardening of the operational toolchain and allow for the aggressive line-reduction sweep requested. The legacy template in docs/ must be deprecated to ensure the stability of the task dashboard and the integrity of the Filing Doctrine.

Works cited

1. OPEN-main-b7a7a2eb.txt
2. accessed December 31, 1969, uploaded:llms-full.txt